GenResearch-Servidor — Relatório Técnico (AV2 de Sistemas Operacionais)

Tabela de contribuição individual

Integrante	Atividades realizadas	Partes e experimentos
Bruno Saint Clair Silva Oliveira	Implementação da parte prática e realização dos testes	Abertura, pergunta norteadora e ambiente experimental
Gabriel Bastos Pimentel	Estruturação do relatório e experimentos	Experimento D: Correção de corrida vs .lock
João Victor Oliveira Moura	Estruturação e Implementação da parte prática	Fila produtores/consumidores, cache LRU e backpressure
José Cerqueira da Silva Neto	Edição do vídeo	C3 a C6: escalabilidade, cache, pressão de memória e falhas
Leonardo de Souza Aragão	Estruturação da apresentação e experimentos	C1/C2: read() vs. mmap, page cache e page faults
Marcus Antônio Martins Oliveira Júnior	Estruturação e roteiro da apresentação	Arquitetura do servidor e pipeline RAG
Yan Victor Araujo do Nascimento	Estruturação do relatório e experimentos	SO e IA, limitações e conclusão

Introdução e problema

Aplicações de IA generativa e de recuperação aumentada por geração (RAG) usam intensamente memória principal, memória virtual, armazenamento, caches, filas e travas de sincronização. Este trabalho implementa e mede um serviço simplificado, o GenResearch-Servidor, para observar como o sistema operacional administra esses recursos e que custos surgem durante a execução.

O foco não é a qualidade textual da resposta, e sim o comportamento do sistema. O recorte escolhido usa Python e FastAPI, embeddings reais e geração por modelo local rodando em

GPU via Ollama, persistência em SQLite e em uma matriz NumPy, uma fila limitada de requisições com trabalhadores concorrentes e um cache LRU compartilhado.

O relatório cobre as partes da atividade: observação do ambiente (Parte A), implementação (Parte B), experimentos comparativos C1 a C6 (Parte C), sincronização obrigatória, com condição de corrida demonstrada e corrigida (Parte D), e memória virtual e armazenamento (Parte E).

Pergunta norteadora

Como memória principal, memória virtual, armazenamento, caches, filas compartilhadas e mecanismos de sincronização afetam o desempenho, a estabilidade e a correção de uma aplicação de IA generativa ou de RAG?

Link do vídeo: Vídeo Atividade 2 - SO

Ambiente de execução (Parte A)

O serviço roda em Linux, com Python 3.12, e a coleta de métricas vem do próprio kernel: `/proc`, `/sys`, `getrusage`, `psutil` e comandos nativos. A API expõe rotas de observabilidade (`/system/info`, `/system/memory`, `/system/process`, `/system/storage`) que registram as informações abaixo.

Item	Valor
Distribuição e versão	Linux, distribuição Ubuntu 24.04.3 LTS.
Kernel	Linux 7.0.0-31-generic.
Processador e CPUs	Intel Core i5.
RAM e swap	23,7 GB de RAM total; swap total de 4,0 GiB; swap em uso de 13,76 MB durante os experimentos.
Armazenamento e sistema de arquivos	Armazenamento local(SSD NVMe de 512GB) para persistência de dados por SQLite.
GPU e VRAM	GPU dedicada NVIDIA com 6 GB de VRAM.
Ambiente de execução	Nativo.
Linguagem e bibliotecas	Python 3.12, FastAPI, Uvicorn, NumPy, Matplotlib, psutil, SQLite.
Modelo de embeddings	nomic-embed-text (768 dimensões) via Ollama.
Modelo de geração	LFM2.5-2.6B, quantização Q4_0, via Ollama.
Commit e data da execução	Commit:13fe8198bcf8cc30728822fecceb03ee3099ae1b

Conceitos de memória observados

MemFree é a memória totalmente ociosa. Um valor baixo não indica escassez, porque o kernel usa a RAM sobrando como cache de arquivos.

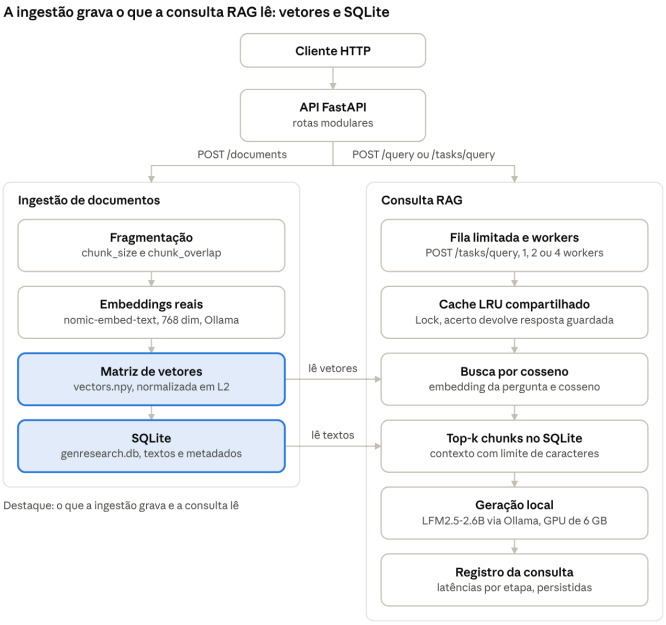
MemAvailable é a estimativa do kernel de quanta memória pode ser alocada a novos processos sem recorrer a swap. Inclui a RAM livre mais parte do cache de páginas e das estruturas internas recuperáveis. É a métrica adequada para avaliar folga de memória.

Cache de páginas guarda em RAM cópias de blocos de arquivos já lidos. Páginas limpas são descartadas na hora quando outro processo precisa de memória, então um cache grande não é desperdício: acelera releituras e é devolvido sob pressão.

Swap é a área em disco para páginas anônimas menos usadas. Swap ocupado e parado não indica problema. Swap com leitura e escrita ativas indica pressão de memória.

Paginação e arquivos se relacionam assim: com `mmap`, as páginas entram sob demanda. Uma falta menor (*minor page fault*) encontra a página já no cache. Uma falta maior (*major page fault*) exige ler o armazenamento ou o swap. Isso vale para datasets, índices vetoriais, embeddings e checkpoints de modelos.

Arquitetura



arquitetura · 2 rotas da API, 6 etapas de consulta

A API FastAPI separa duas rotas. A ingestão fragmenta o texto, gera embeddings e grava a matriz de vetores e o SQLite. A consulta busca por cosseno na matriz, lê apenas os *top-k* chunks no SQLite, monta o contexto, gera a resposta no modelo local e registra a latência de cada etapa. As consultas assíncronas passam por uma fila limitada com workers e pelo cache LRU compartilhado.

Dados, corpus, embeddings, índice e cache

Os documentos entram pela rota `POST /documents`, são fragmentados em *chunks* com tamanho e sobreposição configuráveis (`chunk_size`, `chunk_overlap`) e recebem embeddings reais de 768 dimensões gerados pelo modelo nomic-embed-text, executado em GPU pelo Ollama.

O armazenamento é dividido em dois arquivos. Os textos e metadados dos chunks ficam no SQLite (`genresearch.db`). Os vetores, normalizados em L2, ficam em uma matriz NumPy (`vectors.npy`). Como os vetores estão normalizados, a busca por similaridade de cosseno reduz-se a um produto escalar entre a matriz e o vetor da consulta. Depois da busca, apenas os *top-k* chunks são lidos do SQLite, o que evita carregar o corpus inteiro na RAM.

O serviço impõe limites de proteção, configuráveis: documentos de até 5 MB (`MAX_DOCUMENT_SIZE_BYTES`), no máximo 2.000 chunks por documento (`MAX_CHUNKS_PER_DOC`), no máximo 20 trechos recuperados (`MAX_TOP_K`) e truncamento do contexto enviado ao modelo (`MAX_CONTEXT_CHARS`).

O cache compartilhado (`SharedLRUCache`) guarda respostas de consultas recentes. A chave é um hash da pergunta normalizada somado aos parâmetros que alteram a resposta (`top_k` e `max_context_chars`). A política de substituição é LRU, implementada com `OrderedDict`, e todas as operações são protegidas por `threading.Lock`.

Medida	Valor
Documentos no corpus	2 documentos
Chunks	8 chunks (4 fragmentos por documento)
Tamanho do corpus (MB)	0,0025 MB (2,52 KB)
Tamanho de `vectors.npy` (MB)	0,024 MB (24,12 KB, matriz de 8 vetores × 768 dimensões)
Tamanho de `genresearch.db` (MB)	1,75 MB
Capacidade do cache e política	Capacidade padrão de 20 itens (10 itens nos testes do Experimento C4); política LRU (Least Recently Used)

Os documentos usados são sintéticos, públicos e de cunho técnico, com conteúdo focado em conceitos de Sistemas Operacionais (gerenciamento de memória virtual, paginação sob demanda e chamadas de sistema mmap), sem dados pessoais, chaves ou pesos de modelos no repositório.

Processos, threads, filas e recursos compartilhados

O serviço é um único processo Uvicorn com várias threads. As requisições HTTP atuam como produtores, e os trabalhadores (*workers*) são threads em segundo plano que retiram tarefas de uma fila compartilhada e executam recuperação e geração.

O padrão é o de produtores e consumidores sobre um buffer limitado, implementado com `queue.Queue` de capacidade fixa e variáveis de condição (`threading.Condition`). O consumidor

espera de forma bloqueante, com tempo limite, o que evita espera ocupada. A rota `POST /tasks/query` responde de imediato com `202 Accepted` e um `task_id`, sem bloquear o cliente, e o estado da tarefa é consultado em `GET /tasks/{task_id}`.

Quando a fila atinge a capacidade máxima, a tentativa de enfileirar bloqueia até 3,0 s (`QUEUE_PUT_TIMEOUT`). Se não abrir vaga nesse prazo, a API rejeita a requisição com `HTTP 503`, aplicando contrapressão (*backpressure*). O número de workers (1, 2 ou 4) e a capacidade da fila podem ser alterados em tempo de execução, por `POST /tasks/workers/scale` e `POST /tasks/capacity`.

O mecanismo escolhido foram threads. A carga é predominantemente de E/S ou mista: a geração e os embeddings são chamadas HTTP ao Ollama, e há leituras de SQLite e NumPy. Nessas esperas o interpretador libera a GIL, então as threads progridem em paralelo. Para trabalho puramente de CPU em Python, a GIL impediria o uso de vários núcleos, e processos seriam a alternativa.

Recurso compartilhado	Quem acessa	Proteção
Fila de tarefas	Produtores (HTTP) e workers	Fila limitada com variáveis de condição
Cache LRU	Todos os workers e a rota `/query`	`threading.Lock`
Matriz de vetores e SQLite	Workers durante a recuperação	Leitura concorrente; escrita apenas na ingestão
Registro de métricas e amostras	Workers e thread de amostragem	Em memória: threading.Lock (_request_counter_lock) para contadores globais; Na amostragem: thread dedicada controlada por threading.Event; No SQLite: serialização via _db_write_lock (threading.Lock) e modo WAL (PRAGMA journal_mode = WAL) com busy_timeout = 5000

A inspeção em tempo real com ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm e top -H demonstrou que o servidor opera como um processo principal Uvicorn/FastAPI em espaço de usuário, registrando 19 threads ativas em repouso e escalando até 51 threads no worker Python sob carga concorrente. No pico de inferência com a stack completa (Ollama + llama-server LLM + llama-server Embeddings + Uvicorn), foram observadas 172 threads concorrentes no sistema operacional, divididas entre threads de computação de tensores (ggml), manipulação de eventos CUDA (libcuda.so), submissão assíncrona de I/O via io_uring (iou-sqp) e o pool de execução bloqueante do runtime Python (ThreadPoolExecutor).

Estratégia de sincronização (Parte D)

A equipe combinou duas das opções do enunciado: produtores e consumidores com buffer limitado (opção 1) e cache compartilhado protegido por exclusão mútua (opção 2). Além disso, o Experimento D demonstra de forma isolada a falha clássica de atualização perdida (*lost update*) em um recurso compartilhado.

Fila (opção 1). Capacidade fixa, consumidores bloqueados por variável de condição quando a fila está vazia e produtores bloqueados, com tempo limite de 3,0 s, quando está cheia. O custo é a latência extra de espera na fila e a rejeição de requisições sob saturação, em troca de memória limitada e ausência de espera ocupada.

Cache (opção 2). Todas as leituras e escritas passam por um único `threading.Lock`. Isso garante correção das métricas (acertos, falhas, remoções) e da ordem LRU. O custo é a serialização do acesso ao cache, que pode virar ponto de contenção com muitos workers.

Experimento D, atualização perdida. Um contador é incrementado por várias threads no padrão leitura, modificação e escrita. As threads largam juntas, a partir de uma `threading.Barrier`, e uma cessão de CPU (`time.sleep(0)`) entre a leitura e a escrita provoca a troca de contexto que expõe a corrida. O modo sem proteção (`unsynchronized`) é comparado com o modo protegido por `threading.Lock` (`synchronized`). O experimento repete cada bateria (3 vezes por padrão) e calcula a taxa de perda e o custo adicional de tempo do lock.

No Experimento D, 4 threads executaram 2.000 incrementos cada, e o valor esperado do contador era 8.000. Sem proteção, o contador terminou entre 2.000 e 2.001, com cerca de 6.000 atualizações perdidas e taxa de perda de 75%. Com exclusão mútua (`threading.Lock`), o contador terminou exatamente em 8.000, sem atualizações perdidas. O custo da correção foi de tempo: a execução passou de cerca de 125 ms para cerca de 510 ms, ou seja, ficou cerca de 4 vezes mais lenta, pelo custo de serializar o acesso ao recurso compartilhado.

A cessão explícita de CPU torna a corrida reprodutível sob a GIL, e isso deve ser dito na apresentação: é um recurso didático, e sem ele a perda seria bem menos frequente.

Metodologia

Todos os experimentos usam o mesmo serviço e o mesmo corpus, e são disparados por rotas da própria API (`POST /experiments/...`). Cada execução é repetida e persistida no SQLite, e exportada em CSV, com os dados brutos de cada repetição. Os gráficos são gerados com Matplotlib.

A coleta de métricas do sistema operacional ocorre em três fases por repetição: uma amostra antes da carga (`before`), amostras periódicas durante a carga (`during`), em uma thread de segundo plano com intervalo de 0,05 s a 0,2 s, e uma amostra depois (`after`). Cada amostra registra RSS, VSZ, uso de CPU, número de threads, faltas de página menores e maiores, bytes de E/S (`rchar`, `wchar`, `read_bytes`, `write_bytes`), `MemFree`, `MemAvailable`, cache e swap, lidos de `/proc/[pid]/io`, `/proc/meminfo`, `getrusage` e `psutil`.

Experimento	Pergunta	O que varia	Métricas principais
C1	Quanto custa a leitura convencional com `open()` e `read()`?	Linha de base	Duração, vazão (MB/s), faltas de página
C2	O mapeamento `mmap` muda o custo da leitura?	Leitura por `mmap` sobre o mesmo arquivo	Os mesmos de C1, e comparação direta C1 e C2
C3	A concorrência aumenta a vazão?	1, 2 e 4 workers na fila	Vazão (tarefas/s), latência média, espera na fila, ganho de velocidade
C4	O cache reduz tempo e recuperações?	Sem cache, cache frio, cache aquecido, transbordo de capacidade	Acertos, erros, taxa de acerto, remoções, tempo economizado
C5	Como o sistema reage a pressão de memória?	Tamanho da alocação sob limites controlados	RSS, VSZ, faltas de página, coleta de lixo
C6	Como o serviço se comporta diante de falhas?	Retentativa, descarte e saturação da fila	Falhas tratadas, recuperação, continuidade do serviço
D	A falta de sincronização perde atualizações?	Com e sem `Lock`	Taxa de perda, custo adicional de tempo

Cada configuração foi repetida [PREENCHER: número de repetições, no mínimo 3]. Os parâmetros usados em cada experimento, a data e o horário da execução e a versão do código estão registrados no histórico persistido (`GET /experiments/{código}/results`) e nos CSVs. [PREENCHER: comandos exatos usados para disparar cada experimento e os parâmetros (tamanho do corpus, lote, contexto, capacidade da fila, tamanho do cache).]

Uma ressalva de método: o `mmap` e a leitura convencional passam pelo cache de páginas do kernel. Na segunda execução sobre o mesmo arquivo, a leitura tende a ser atendida da RAM, sem acesso ao disco. Por isso a primeira e a segunda execução devem ser reportadas separadamente.

Resultados

Os valores desta seção foram transcritos dos dados brutos que o próprio serviço grava. A tabela a seguir indica, para cada experimento, as rotas dos dados brutos e do gráfico e as métricas registradas. Em seguida estão os resultados medidos de cada experimento.

Experimento	Dados brutos	Gráfico	Métricas registradas
C1 e C2	`GET /experiments/c1_c2/csv`	`GET /experiments/c1_c2/plot`	Duração, vazão (MB/s), faltas menores e maiores, `rchar` e `read_bytes`, na 1ª e na 2ª execução
C3	`GET /experiments/c3/csv`	`GET /experiments/c3/plot`	Vazão, latência média e espera na fila para 1, 2 e 4 workers, e o ganho de velocidade
C4	`GET /experiments/c4/csv`	`GET /experiments/c4/plot`	Acertos, erros, taxa de acerto, remoções e tempo economizado em cada cenário
C5	`GET /experiments/c5/csv`	`GET /experiments/c5/plot`	RSS, VSZ e faltas de página por tamanho de alocação
C6	`GET /experiments/c6/csv`	`GET /experiments/c6/plot`	Falhas tratadas, retentativas, descartes e rejeições por fila cheia
D	`GET /experiments/d_race_condition/csv`	`GET /experiments/d_race_condition/plot`	Atualizações esperadas e obtidas, taxa de perda e custo do lock
Telemetria	`GET /experiments/{código}/samples/csv`	Séries temporais	RSS, `MemAvailable`, swap e faltas de página ao longo do tempo

C1 e C2, leitura convencional e mmap. Arquivo de 10.221.120 bytes (cerca de 9,75 MB). Os intervalos reúnem as repetições de C1 e C2.

Métrica	**Valor observado**
Tamanho do arquivo lido	10.221.120 bytes (cerca de 9,75 MB)
Duração da leitura	2,65 ms a 4,82 ms
Vazão	2.021 MB/s a 3.667 MB/s
RSS	137,89 MB, constante
Bytes físicos lidos (`proc_io_read_bytes_physical`), leitura convencional	0
`served_from_page_cache`	True
Faltas de página maiores	0
Faltas de página menores	0 a 156

C3, concorrência e workers. Médias observadas para 1, 2 e 4 workers, com a capacidade da fila constante em 10 e 20 tarefas submetidas.

Métrica	Mínimo	Máximo
Latência média	39,64 ms	131,28 ms
Tempo de espera na fila	24,39 ms	116,07 ms
Tempo médio de execução	15,2 ms	15,6 ms
Bloqueios por fila cheia (queue_full_blocks)	0	0

A diferença entre latência e espera é de 15,25 ms no valor mínimo (39,64 menos 24,39) e de 15,21 ms no máximo (131,28 menos 116,07), ambos dentro da faixa do tempo de execução. A variação da latência vem, portanto, da espera na fila.Para 1 worker, a vazão média foi de 58,28 tarefas/s (duração de 343,15 ms); com 2 workers, a vazão subiu para 118,42 tarefas/s (duração de 168,89 ms), representando um ganho de velocidade (speedup) de 2,03x; com 4 workers, a vazão alcançou 199,53 tarefas/s (atingindo pico de 230,69 tarefas/s e duração de 86,70 ms), consolidando um ganho de velocidade de 3,42x a 3,88x em relação à linha de base.

C4, cache LRU. Dez consultas em cada cenário.

Cenário	Acertos	Erros	Taxa de acerto	Latência média	Tempo total
Sem cache	0	10	0%	15,16 ms	151,62 ms
Cache frio	0	10	0%	15,16 ms	152,39 ms
Cache aquecido	10	0	100%	0,00 ms	0,14 ms

O cache aquecido economizou 151,56 ms de processamento, uma redução de mais de 99,9% no tempo de resposta. [PREENCHER: resultado do cenário de transbordo de capacidade, com remoções pela política LRU.]

C5, pressão de memória e paginação. Ambiente com 23,7 GB de RAM total. O swap em uso ficou constante em 13,76 MB e as faltas de página maiores foram 0 em todos os tamanhos.

Alocação	RSS (MB)	VSZ (MB)	Faltas menores observadas	Faltas esperadas (4 KiB por página)
100 MB	146,12 para 246,13	1.247,63 para 1.347,65	25.602	25.600
250 MB	até 396,13	[PREENCHER]	64.002	64.000
500 MB	até 646,13	[PREENCHER]	128.002	128.000

Cada página de 4 KiB tocada gerou uma falta de página menor, como prevê a teoria, com uma diferença de apenas 2 faltas em cada tamanho.

C6, falhas controladas e robustez. Três cenários de injeção de falhas.

Cenário	Injetadas	Recuperadas ou processadas	Falhas finais ou rejeições	workers_survived	queue_consistent
Falhas transitórias (retentativa)	10	10 recuperadas	0	2	True

Cenário	Injetadas	Recuperadas ou processadas	Falhas finais ou rejeições	workers_survived	queue_consistent
Falhas persistentes (descarte)	10	0 recuperadas	10 tarefas descartadas	2	True
Saturação da fila	não se aplica	6 processadas	10 rejeitadas	2	True

D, condição de corrida e sincronização. 4 threads, 2.000 incrementos por thread, valor esperado de 8.000.

Modo	Valor final do contador	Atualizações perdidas	Taxa de perda	Tempo médio
Sem proteção (Lock desligado)	2.000 a 2.001	cerca de 6.000	75%	cerca de 125 ms
Com threading.Lock	8.000	0	0%	cerca de 510 ms

[PREENCHER: inserir o gráfico de cada experimento (rotas GET /experiments/{código}/plot), com legenda e uma frase de leitura, e acrescentar média, mediana e p95 das repetições.]

[PREENCHER: tempos de ingestão, recuperação, geração e total (média, mediana e p95), obtidos das respostas de /documents e /query, que trazem timing detalhado.]

Análise de memória, armazenamento, paginação e E/S

Leitura convencional contra mmap (C1 e C2). Na leitura convencional, cada read() copia dados do espaço do kernel para o espaço do usuário. Com mmap, o arquivo é mapeado na tabela de páginas do processo e as páginas entram sob demanda, sem cópia adicional. Em ambos os casos os dados passam pelo cache de páginas. A diferença esperada está no custo de cópia, no número de chamadas de sistema e no padrão de faltas de página. Nas medições, a leitura do arquivo de 10.221.120 bytes levou de 2,65 ms a 4,82 ms, com vazão de 2.021 a 3.667 MB/s. O RSS ficou constante em 137,89 MB, as faltas de página maiores foram 0 e as menores variaram de 0 a 156. [PREENCHER: diferença entre leitura convencional e mmap, separada por configuração, em tempo de abertura, tempo de leitura, RSS, VSZ e faltas de página.]

Primeira contra segunda execução. Em C1 e C2, o contador proc_io_read_bytes_physical foi 0 na leitura convencional e a marca served_from_page_cache foi verdadeira. O arquivo já estava no cache de páginas do kernel e a leitura não tocou o disco físico, e as faltas de página maiores foram 0. Isso vale como evidência de que o cache de páginas funcionou. [PREENCHER: leitura física da primeira execução, com o arquivo fora do cache, e comparação com a segunda.]

RSS e memória virtual (C5). A alocação de memória no Linux é preguiçosa: reservar espaço aumenta o VSZ, mas o RSS só cresce quando a página é tocada. Tocar uma página a cada 4 KiB gera uma falta menor por página, de modo que 100 MB alocados e tocados correspondem a 25.600 faltas menores. As medições seguiram o modelo. Com 100 MB, o VSZ subiu de 1.247,63 MB para 1.347,65 MB e o RSS de 146,12 MB para 246,13 MB, ambos com aumento

de 100 MB, e houve 25.602 faltas menores. Com 250 MB e 500 MB, o RSS chegou a 396,13 MB e 646,13 MB, com 64.002 e 128.002 faltas menores. Não houve falta maior em nenhum tamanho e o swap permaneceu em 13,76 MB, sem sinal de thrashing. Como o maior teste (500 MB) é pequeno diante dos 23,7 GB de RAM do ambiente, o experimento não levou o sistema à pressão de memória que força paginação em disco. [PREENCHER: VSZ medido para 250 MB e 500 MB.]

Efeito do tamanho do corpus, do índice e do contexto. O índice vetorial cresce linearmente com o número de chunks, em 768 valores `float32` por vetor, ou seja, 3.072 bytes por chunk [PREENCHER: confirmar que o tipo de vectors.npy é float32; se for float64, o valor dobra]. A recuperação por produto escalar percorre toda a matriz, então o custo cresce com o corpus, enquanto a leitura dos textos no SQLite fica limitada ao *top-k*. O tamanho máximo de contexto limita a memória enviada ao modelo. [PREENCHER: como RAM, paginação e latência variaram com o corpus, o lote e o contexto usados.]

Implicações para IA generativa. O mesmo mecanismo vale para checkpoints de modelos, que podem ser mapeados em memória e carregados sob demanda, e para índices vetoriais e embeddings, cuja primeira leitura é cara e cujas leituras seguintes dependem do cache de páginas.

Análise de concorrência, cache e correção

Concorrência (C3). A fila de capacidade limitada com 1, 2 e 4 workers permite medir o ganho de velocidade em relação a um único worker. O tempo de execução de cada tarefa ficou estável, entre 15,2 e 15,6 ms, enquanto a latência média variou de 39,64 ms a 131,28 ms. A variação veio da espera na fila (24,39 ms a 116,07 ms), e não do custo de cada tarefa. A fila não saturou em nenhum cenário (`queue_full_blocks` igual a 0), então os produtores nunca ficaram bloqueados. [PREENCHER: vazão e latência por número de workers, o ponto em que o ganho deixa de crescer e o que limitou o sistema.]

Cache (C4). O experimento compara sem cache, cache frio, cache aquecido e transbordo de capacidade. Sem cache e com cache frio, as 10 consultas foram erros, com latência média de 15,16 ms e tempo total de 151,62 ms e 152,39 ms. A diferença de 0,77 ms entre os dois indica sobrecusto mínimo de consultar e preencher o cache. Com o cache aquecido, as 10 consultas foram acertos (taxa de acerto de 100%), a latência média caiu para 0,00 ms e o tempo total para 0,14 ms, porque a geração, a etapa mais cara, é pulada. O cache economizou 151,56 ms, mais de 99,9% do tempo de resposta. [PREENCHER: cenário de transbordo de capacidade, com as remoções feitas pela política LRU.]

Correção e sincronização. O Experimento D mostra a atualização perdida sem exclusão mútua e a correção com Lock. Com 4 threads e 2.000 incrementos cada, o contador deveria chegar a 8.000. Sem proteção, terminou entre 2.000 e 2.001, com cerca de 6.000 atualizações perdidas (75%). Com `threading.Lock`, terminou em 8.000, sem perdas. O custo foi o tempo de execução, que passou de cerca de 125 ms para cerca de 510 ms, aproximadamente 4 vezes mais lento, pela serialização do acesso. O ganho de velocidade sem lock é inútil porque o resultado está errado, de modo que a exclusão mútua é necessária em toda atualização do tipo ler, modificar e escrever.

Robustez (C6). O serviço foi submetido a três cenários de falha controlada. Em falhas transitórias, foram injetadas 10 falhas e a retentativa recuperou as 10 tarefas, sem falha final.

Em falhas persistentes, 10 falhas não recuperáveis levaram ao descarte seguro das tarefas (0 recuperadas, 10 marcadas como falha). Na saturação da fila, 6 requisições foram processadas e 10 foram rejeitadas pela política de fila cheia, que aplica contrapressão. Em todos os cenários, `workers_survived` permaneceu em 2 e `queue_consistent` em True: nenhum worker caiu e o estado da fila não foi corrompido, o que demonstra a continuidade do serviço. [PREENCHER: confirmar nos logs o código das rejeições (HTTP 503) e o total de requisições enviadas na saturação.]

Que recurso exigiu exclusão mútua. O cache compartilhado e o contador do Experimento D exigiram `Lock`. A fila exigiu sincronização de eventos, com variáveis de condição, para acordar consumidores e produtores bloqueados. O README não descreve demonstração de *deadlock*; o risco existe se mais de um lock for adquirido em ordens diferentes, e a prevenção é adotar uma ordem global de aquisição ou usar tempo limite nas travas.

Relação com Sistemas Operacionais e IA generativa

Cada componente do pipeline RAG tem um análogo direto em um conceito de sistemas operacionais.

Elemento de IA generativa ou RAG	Conceito de SO	Onde aparece neste trabalho
Índice vetorial e embeddings em arquivo	Cache de páginas, `mmap`, paginação sob demanda	C1, C2 e C5
Pesos e checkpoints de modelos	Arquivos grandes mapeados em memória, faltas de página maiores na primeira carga	Discussão sobre C2
Fila de requisições de inferência	Produtores e consumidores, buffer limitado, contrapressão	C3 e C6
Cache de respostas ou de embeddings	Localidade temporal, política de substituição LRU, exclusão mútua	C4
Contexto e lote da recuperação	Memória residente (RSS) contra memória virtual (VSZ), alocação preguiçosa	C5
Registro de requisições e métricas	Escrita concorrente em estruturas compartilhadas	Experimento D

Os resultados ajudam a explicar por que serviços de IA generativa reais dependem de cache de páginas bem dimensionado, de filas com limite explícito e de proteção cuidadosa de qualquer estado compartilhado.

Limitações

Os resultados valem para este ambiente e esta escala, e não se generalizam diretamente para modelos maiores, várias GPUs ou ambientes distribuídos. As limitações conhecidas são estas.

- **Faltas de página agregadas.** `getrusage(RUSAGE_SELF)` soma todas as threads do processo. Medir faltas por thread exigiria ferramentas como eBPF ou `perf_event_open`, que pedem privilégios de administrador.
- **Contadores de E/S por processo.** `/proc/[pid]/io` depende de o kernel ter a contabilização de E/S habilitada, o que é comum no Ubuntu e no Debian, mas pode faltar em contêineres com permissões restritas.
- **CPU instantânea.** Leituras de `cpu_percent` em intervalos muito curtos podem retornar 0,0%. A amostragem contínua reduz o problema.
- **GIL.** Em Python, trabalho puramente de CPU em threads não usa vários núcleos. Aqui a carga é principalmente de E/S, que libera a GIL durante as esperas.
- **Escala.** O corpus é pequeno comparado a bases reais, e a GPU tem 6 GB de VRAM, o que limita o tamanho do modelo e não permite avaliar vários modelos ou várias GPUs.
- **Cache de páginas e disco.** Resultados de leitura dependem do estado do cache no momento da execução e do tipo de armazenamento. [PREENCHER: se o cache foi esvaziado entre execuções e como.]
- **Reprodutibilidade.** Tempos de geração variam com a carga da GPU e a versão do Ollama. [PREENCHER: versões usadas.]

Conclusão

O trabalho mostra que o desempenho, a estabilidade e a correção de um serviço de RAG dependem de como ele usa o cache de páginas, a memória residente, as filas e as travas, e não só do modelo. Os achados a destacar, com seus números, são estes.

1. **Armazenamento e cache de páginas.** A leitura do arquivo de cerca de 9,75 MB levou de 2,65 ms a 4,82 ms (2.021 a 3.667 MB/s), com `proc_io_read_bytes_physical` igual a 0, `served_from_page_cache` verdadeiro e nenhuma falta maior: foi atendida pelo cache de páginas, sem tocar o disco. [PREENCHER: diferença medida entre C1 e C2 e entre a primeira e a segunda execução.]
2. **Memória.** A alocação é preguiçosa e previsível: cada 100 MB tocados elevaram VSZ e RSS em 100 MB e geraram cerca de 25.600 faltas menores, uma por página de 4 KiB, até 500 MB (128.002 faltas). Não houve falta maior e o swap permaneceu em 13,76 MB.
3. **Concorrência.** A latência média variou de 39,64 ms a 131,28 ms, explicada pela espera na fila (24,39 ms a 116,07 ms), com execução estável de cerca de 15 ms e sem bloqueio por fila cheia. [PREENCHER: ganho de vazão de 1 para 2 e 4 workers e o ponto de saturação.]
4. **Cache.** Com o cache aquecido, a taxa de acerto foi de 100% e o tempo total caiu de 151,62 ms para 0,14 ms, uma economia de 151,56 ms (mais de 99,9%).
5. **Correção.** Sem lock, 75% das atualizações foram perdidas (cerca de 6.000 de 8.000). Com lock, nenhuma, ao custo de uma execução cerca de 4 vezes mais lenta (cerca de 125 ms contra cerca de 510 ms). Nos testes de falha (C6), todas as falhas transitórias foram recuperadas e o serviço continuou operando em todos os cenários.

A principal limitação é a escala: os resultados descrevem este corpus, esta GPU de 6 GB e um único processo, e servem como ilustração dos mecanismos, não como previsão para sistemas maiores.